Introduction, Environnement et Syntaxe de base

Partie 1 : Introduction et mise en place de l'environnement

1.1 Genèse et philosophie du langage Go

Créé chez Google en 2007 (annoncé en 2009) par trois figures historiques de l'informatique :

- **Ken Thompson** (co-créateur d'Unix, du langage B et d'UTF-8)
- **Rob Pike** (co-créateur d'UTF-8, Plan 9)
- **Robert Griesemer** (moteur V8 JavaScript, HotSpot JVM)

Pourquoi Go a-t-il été créé ?

Face à des bases de code de plusieurs millions de lignes en C++, Java et Python, Google faisait face à trois goulets d'étranglement :

1. **Temps de compilation trop longs** (le C++ nécessitait parfois des dizaines de minutes sur des clusters distribués).
2. **Complexité de la programmation concurrente** à l'ère des processeurs multicœurs et du Cloud (les threads OS sont lourds et la synchronisation par verrous est source de bugs critiques).
3. **Explosion de la complexité des langages** (C++20 dépasse les 90 mots-clés, multipliant les styles de code incompatibles au sein d'une même équipe).

Les piliers de conception de Go

- **Simplicité radicale :** Seulement **25 mots-clés** (contre 32 en C89). Une seule façon de faire les choses pour garantir que tout développeur puisse lire le code d'un autre instantanément.
- **Compilation native ultra-rapide :** Pas de machine virtuelle (JVM/CLR). Go compile directement en code machine dans un **binaire unique statique** embarquant son propre *runtime* (allocateur mémoire, Garbage Collector, ordonnanceur de goroutines).
- **Concurrence native (Modèle CSP) :** Intégrée dans la syntaxe même du langage via les *goroutines* et les *channels*.

1.2 Architecture de l'environnement et Go Modules

Fin du `$GOPATH`, place aux Go Modules

Historiquement (avant Go 1.11), tout le code Go devait résider dans un dossier unique (`$GOPATH/src`). **Cette approche est aujourd'hui obsolète.** Depuis Go 1.16+, on utilise exclusivement les **Go Modules**, qui permettent de créer un projet dans n'importe quel répertoire du système.

Initialiser un projet

Tout projet Go commence par la création d'un module :

```shell
mkdir mon-projet && cd mon-projet
go mod init github.com/mon-org/mon-projet
```

Cette commande génère un fichier `go.mod` à la racine :

- **`go.mod`** : Définit le chemin canonique du module, la version minimale de Go requise et la liste exacte des dépendances avec leurs versions sémantiques.
- **`go.sum`** : Créé automatiquement dès qu'une dépendance externe est téléchargée. Il contient les empreintes cryptographiques (hashes SHA-256) des dépendances pour garantir l'intégrité de la chaîne logistique (*supply chain security*).

1.3 La chaîne d'outils CLI intégrée (Tooling natif)

Contrairement au C/C++ (qui nécessite Make, CMake, ClangFormat, Valgrind, GTest) ou à JavaScript (Webpack, ESLint, Jest, Prettier), Go intègre tous les outils standards directement dans son binaire `go` :

Commande	**Rôle et utilisation**
`go run main.go`	Compile en mémoire temporaire et exécute immédiatement le programme (idéal en développement).
`go build`	Compile le projet et produit un exécutable binaire natif dans le dossier courant.
`go fmt ./...`	Formate automatiquement tout le code source selon le standard officiel et non négociable de

Commande	Rôle et utilisation
	Go (indentations par tabulations, placement des accolades).
`go vet ./...`	Analyseur statique natif qui détecte les constructions suspectes ou boguées que le compilateur tolère (ex: mauvais formatage dans `Printf`, copie accidentelle d'un verrou).
`go test ./...`	Exécute tous les fichiers de tests unitaires (`*_test.go`) du projet.
`go mod tidy`	Nettoie le fichier `go.mod` : télécharge les dépendances manquantes trouvées dans le code et supprime celles qui ne sont plus utilisées.

La Cross-Compilation native (Compilation croisée)

En C, compiler un binaire Linux ARM64 depuis un poste Windows ou macOS nécessite une chaîne de compilation croisée complexe (*toolchain*). En Go, le compilateur embarque nativement toutes les cibles :

```shell
# Compiler un binaire pour un serveur Linux (x86_64) depuis macOS ou Windows :
GOOS=linux GOARCH=amd64 go build -o mon-serveur-linux main.go

# Compiler pour un Raspberry Pi (ARM64) :
GOOS=linux GOARCH=arm64 go build -o mon-serveur-arm main.go
```

1.4 Anatomie d'un premier programme Go

```go
package main

import (
	"fmt"
	"runtime"
)

func main() {
	fmt.Printf("Bonjour depuis Go (%s/%s) !\n", runtime.GOOS, runtime.GOARCH)
```

```
}
```

Règles strictes du compilateur Go à retenir :

1. **Point d'entrée :** Un exécutable doit obligatoirement appartenir au `package main` et posséder une fonction `func main()` (sans arguments et sans valeur de retour).
2. **Tolérance zéro sur le code mort :** Toute variable locale déclarée mais non lue, ou tout package importé mais non utilisé, provoque une **erreur de compilation bloquante** (et non un simple *warning*).
3. **Absence de point-virgule (`;`) :** Le lexer de Go insère automatiquement les points-virgules en fin de ligne lors de l'analyse lexicale. **Conséquence syntaxique :** l'accolade ouvrante `{` d'une fonction, d'un `if` ou d'un `for` doit **impérativement** se trouver sur la même ligne que la déclaration, sinon le compilateur insère un `;` avant l'accolade !

Partie 2 : Syntaxe de base, types et structures du langage

2.1 Variables, constantes et système de types

Go est un langage à **typage statique et fort**. Il n'existe aucune conversion implicite de type (même entre un `int32` et un `int64`).

Le concept de "Zero Value" (Zéro mémoire poubelle)

En C, déclarer `int x;` sur la pile sans l'initialiser laisse dans `x` les octets résiduels de la RAM (*garbage value*). **En Go, la mémoire non initialisée n'existe pas.** Toute variable déclarée reçoit automatiquement sa *Zero Value* :

- `0` pour tous les types numériques (`int`, `float64`, `byte`...)
- `false` pour le type `bool`
- `""` (chaîne vide, et non `NULL`) pour le type `string`
- `nil` pour les pointeurs, les slices, les maps, les channels, les fonctions et les interfaces.

Syntaxes de déclaration

```Go
// 1. Déclaration explicite (recommandée si on veut partir de la Zero Value)
var compteur int // vaut 0 automatiquement

// 2. Déclaration avec initialisation
```

```go
var message string = "Initialisé"

// 3. Déclaration courte avec inférence de type (uniquement à l'intérieur d'une fonction !)
port := 8080          // Go déduit le type 'int'
actif := true         // Go déduit le type 'bool'
pi := 3.14159         // Go déduit le type 'float64'
```

Types primitifs fondamentaux

- **Entiers signés :** `int8`, `int16`, `int32`, `int64`, et `int` (dont la taille dépend de l'architecture : 64 bits sur les systèmes 64 bits).
- **Entiers non signés :** `uint8`, `uint16`, `uint32`, `uint64`, `uint`, `uintptr`.
- **Flottants :** `float32`, `float64` (norme IEEE-754).
- **Chaînes et caractères (`string`, `byte`, `rune`) :**
  - Une `string` en Go est une séquence **immuable** d'octets (encodée par défaut en UTF-8). Ce n'est pas un tableau terminé par `\0` comme en C : sous le capot, une `string` contient un pointeur vers les données et une longueur `len`.
  - `byte` est un alias strict de `uint8` (représente 1 octet brut ASCII/binaire).
  - `rune` est un alias strict de `int32` (représente un point de code Unicode complet, qui peut occuper de 1 à 4 octets en UTF-8, comme `é` ou `🚀`).

Conversions explicites

```go
Go
var a int = 42
var b float64 = float64(a) // Conversion explicite obligatoire (var b float64 = a ne compile pas)
```

Constantes et l'énumérateur `iota`

Les constantes sont évaluées à la compilation. Go ne possède pas le mot-clé `enum` du C, mais utilise un générateur de séquence appelé `iota` au sein d'un bloc `const` :

```go
Go
type Role int

const (
	RoleInvité Role = iota // 0
	RoleMembre             // 1 (incrémenté automatiquement)
	RoleModérateur         // 2
	RoleAdmin              // 3
)
```

```
// Exemple avancé avec décalage binaire (Flags) :
const (
	Lecture   = 1 << iota // 1 << 0 = 1 (001)
	Ecriture              // 1 << 1 = 2 (010)
	Execution             // 1 << 2 = 4 (100)
)
```

2.2 Structures de contrôle

Conditions `if / else` (avec initialisation locale)

Il n'y a pas de parenthèses autour de la condition, mais les accolades `{}` sont toujours obligatoires. Spécificité très utilisée en Go : on peut déclarer une variable temporaire directement dans l'instruction `if` ; sa portée (*scope*) est alors strictement limitée aux blocs `if` et `else` :

```
Go
if taille := calculerTaille(); taille > 100 {
	fmt.Println("Trop grand :", taille)
} else {
	fmt.Println("Taille valide :", taille)
}
// Ici, la variable 'taille' n'existe plus !
```

La boucle unique : `for`

Les mots-clés `while` et `do...while` n'existent pas en Go. Tout s'écrit avec `for` sous 4 formes :

```
Go
// 1. Forme classique (style C)
for i := 0; i < 10; i++ {
	fmt.Println(i)
}

// 2. Forme conditionnelle (équivalent du 'while' en C)
compteur := 0
for compteur < 5 {
	compteur++
}

// 3. Boucle infinie (équivalent de 'while(1)' en C)
```

```go
for {
	if conditionSortie() {
		break // ou 'continue' pour passer à l'itération suivante
	}
}

// 4. Boucle 'for ... range' (itération sur slices, tableaux, maps, strings, channels)
nombres := []int{10, 20, 30}
for index, valeur := range nombres {
	fmt.Printf("Index %d : %d\n", index, valeur)
}
// Astuce : utiliser l'identifiant vide '_' (Blank Identifier) pour ignorer l'index :
for _, valeur := range nombres {
	fmt.Println(valeur)
}
```

Le `switch / case` modernisé

Contrairement au C, le `switch` en Go **ne tombe pas dans le cas suivant par défaut** : le `break` est implicite à la fin de chaque `case` (si on veut forcer ce comportement, il faut écrire explicitement le mot-clé `fallthrough`).

```go
Go
// Switch avec plusieurs valeurs par cas
switch jour {
case "Samedi", "Dimanche":
	fmt.Println("Week-end")
default:
	fmt.Println("Semaine")
}

// Switch sans expression (remplace élégamment les longues chaînes de if / else if)
score := 85
switch {
case score >= 90:
	fmt.Println("Grade A")
case score >= 75:
	fmt.Println("Grade B")
default:
	fmt.Println("Grade C")
}
```

2.3Structures de données : Arrays, Slices et Maps (Fondamental)

1. Les Tableaux (`Array`) : Rigides et copiés par valeur

En Go, la taille d'un tableau fait partie intégrante de son type : `[4]int` et `[5]int` sont deux types totalement incompatibles. **Différence critique avec le C :** En C, une variable tableau est un pointeur vers le premier élément. **En Go, un tableau est une valeur.** Si vous passez un tableau `[1000000]int` à une fonction, Go copie intégralement les 8 Mo sur la pile ! C'est pourquoi on utilise rarement les Arrays directement.

2. Les `Slices` (`[]T`) : Les vues dynamiques

La `Slice` est la structure séquentielle reine en Go. Sous le capot, une Slice est une toute petite structure (`struct` interne de 24 octets sur architecture 64 bits) contenant **3 champs** :

1. `ptr` : Un pointeur vers un tableau sous-jacent alloué en mémoire.
2. `len` (Longueur) : Le nombre d'éléments actuellement accessibles dans la slice (`len(s)`).
3. `cap` (Capacité) : La taille totale du tableau sous-jacent à partir du pointeur (`cap(s)`).

```Go
// Création d'une slice vide avec make(type, len, cap)
// Pré-allouer la capacité évite les réallocations coûteuses !
s := make([]int, 0, 5) // len = 0, cap = 5

// Ajout d'éléments avec append()
s = append(s, 10, 20, 30) // len = 3, cap = 5
```

Mécanisme de `append` et piège du partage mémoire :

- Tant que `len < cap`, `append` écrit directement dans le tableau sous-jacent existant (ultra-rapide, zéro allocation).
- Si `len == cap`, le tableau sous-jacent est plein : `append` alloue un **nouveau** tableau plus grand (souvent $\times$ 2), copie les anciennes valeurs dedans, et retourne un nouveau descripteur de slice pointant vers cette nouvelle zone mémoire. C'est pour cela qu'on écrit **toujours** `s = append(s, elem)`.
- **Attention au découpage (`slicing`) :** `sousSlice := s[1:3]` ne copie pas les données ! `sousSlice` partage le même tableau sous-jacent que `s`. Modifier `sousSlice[0]` modifiera `s[1]`. Pour faire une vraie copie indépendante, on utilise la fonction native `copy(dest, src)`.

3. Les `Maps` (`map[Clé]Valeur`) : Tables de hachage natives

Une `map` est un dictionnaire clé/valeur non ordonné offrant des accès en $O(1)$.

- **Piège classique :** La *Zero Value* d'une map est `nil`. Lire dans une map `nil` renvoie la *Zero Value* sans planter, mais **écrire dans une map `nil` provoque un crash immédiat (`panic`)**. Il faut toujours l'initialiser avec `make` ou avec une accolade `{}`.

```go
// Initialisation avec make
utilisateurs := make(map[string]int)

// Écriture
utilisateurs["alice"] = 30
utilisateurs["bob"] = 25

// Lecture simple (renvoie 0 si la clé n'existe pas !)
ageInconnu := utilisateurs["charlie"] // vaut 0

// L'idiome du "Comma Ok" : comment savoir si une clé existe vraiment ou si sa valeur est 0 ?
age, existe := utilisateurs["charlie"]
if !existe {
	fmt.Println("Charlie n'est pas dans la map")
} else {
	fmt.Println("Âge de Charlie :", age)
}

// Suppression d'une entrée (sans effet si la clé n'existe pas)
delete(utilisateurs, "bob")
```

2.4 Fonctions, Pointeurs et Gestion Mémoire

Fonctions et retours multiples

En Go, les fonctions sont des citoyens de première classe (on peut les stocker dans des variables ou les passer en paramètres) et elles peuvent retourner **plusieurs valeurs** (ce qui sera la base de toute la gestion des erreurs en Jour 2).

```go
// Fonction retournant un résultat et un booléen de succès
func diviser(dividende, diviseur float64) (float64, bool) {
	if diviseur == 0 {
		return 0, false
	}
	return dividende / diviseur, true
}
```

```
// Fonction variadique (accepte un nombre infini d'arguments, reçus sous forme de slice []int)
func somme(nombres ...int) int {
	total := 0
	for _, n := range nombres {
		total += n
	}
	return total
}
```

Le mot-clé `defer` (Exécution différée garantie)

`defer` permet de planifier l'appel d'une fonction pour qu'elle s'exécute **juste avant que la fonction englobante ne se termine** (quel que soit le `return` emprunté, ou même en cas de crash).

- **Avantage majeur par rapport au C :** Plus besoin de dupliquer les appels `free()` ou `fclose()` avant chaque `return` d'erreur, ni d'utiliser des `goto cleanup`.
- Les appels `defer` sont empilés en mode **LIFO** (Last-In, First-Out : le dernier `defer` déclaré est le premier exécuté à la sortie).

```
Go
func traiterFichier() {
	f := ouvrirRessource()
	defer fermerRessource(f) // Placé juste après l'ouverture : impossible d'oublier de fermer !

	// ... 50 lignes de logique avec plusieurs 'return' anticipés ...
} // <- fermerRessource(f) s'exécute automatiquement ici.
```

Pointeurs, Stack, Heap et *Escape Analysis*

Go possède des pointeurs exactement comme le C (`*T` pour le type pointeur et le déréférencement, `&v` pour prendre l'adresse mémoire d'une variable), avec deux différences majeures :

1. **Pas d'arithmétique de pointeurs :** Impossible d'écrire `ptr++` ou `*(ptr + 4)`. Cela élimine toute une classe de failles de sécurité (*buffer overflows*).
2. **L'*Escape Analysis* du compilateur :** En C, retourner l'adresse d'une variable locale (`int* f() { int x = 5; return &x; }`) est un bug critique (*dangling pointer*) car la pile (*Stack*) est détruite à la fin de `f()`. **En Go, c'est parfaitement valide et courant !**

```go
func creerEntier() *int {
	x := 42
	return &x // 100% sûr en Go !
}
```

Lors de la compilation, l'**Escape Analysis** de Go inspecte le code :

- Si une variable ne sort jamais de la fonction, le compilateur l'alloue sur la **Stack** (allocation et libération instantanées, zéro coût pour le Garbage Collector).
- Si la référence d'une variable "s'échappe" de la fonction (comme `return &x` ci-dessus), le compilateur décide automatiquement de l'allouer sur le **Tas (Heap)**, et le Garbage Collector se chargera de la nettoyer quand plus aucun pointeur ne la référencera.